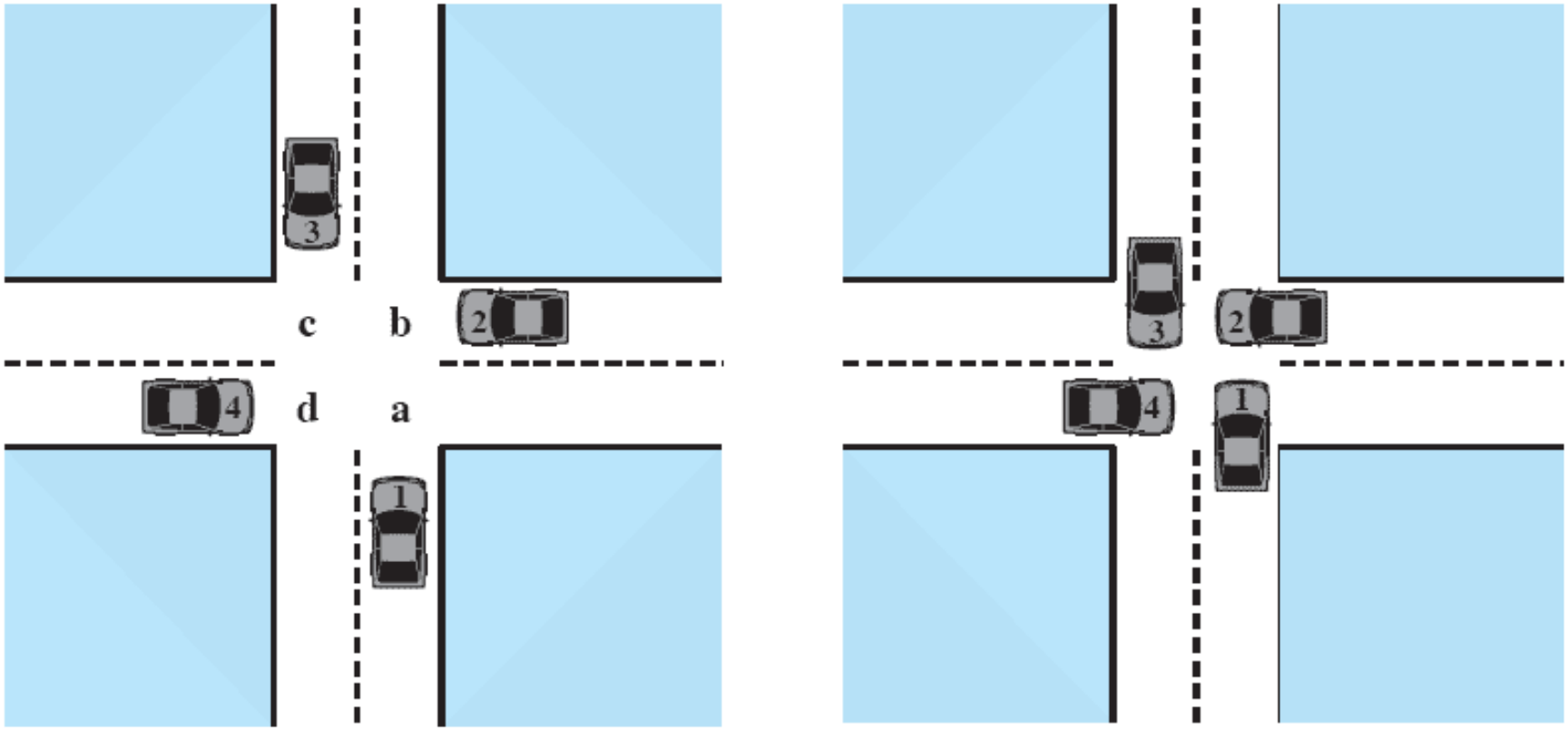
Deadlock

Permanent blocking of a set of processes that either compete for system resources or communicate with each other.



(a) Deadlock possible (b) Deadlock

11455

System Model

- Finite number of resources
- Resources are organized into classes
  - Each class only contain identical resource instances
- Processes compete for accessing resources
- If a process request an instance of a resource class, *any* instance of that class must satisfy the process

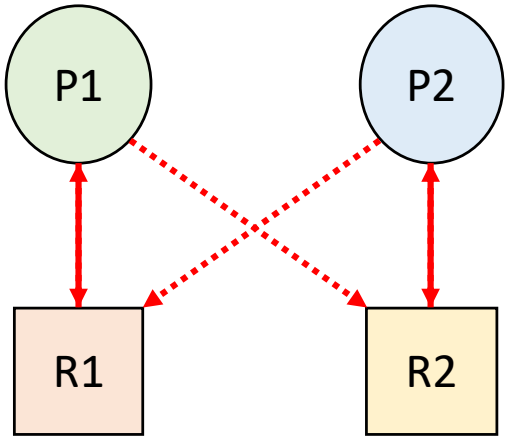
Protocol to Use a Resource

- **Request** — The process either gets an instance of the resource immediately; or waits until one is available (and gets it)
- **Use** — The process can operate on its resource instance
- **Release** — The process releases its resource instance

- Examples: malloc() & free() — open() & close()

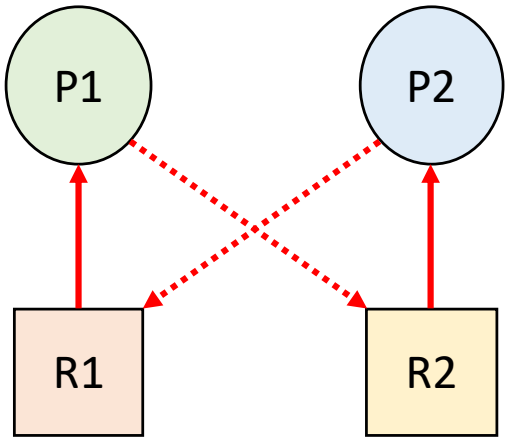
Deadlock

- A set with two or more processes has a deadlocked if:
  1. The processes are blocked (i.e., in the waiting state)
  2. Each blocked process is holding one or more resources
  3. Each blocked process is waiting to acquire a resource held by another process in the set



Deadlock

- Deadlock depends on the dynamics of the execution
- Is difficult to identify and test for deadlocks, which may occur only under certain circumstances



Conditions Necessary for Deadlock

- **mutual exclusion**: only one process can use a resource at a time
- **hold and wait**: a process holding at least one resource is waiting to acquire additional resources which are currently held by other processes
- **no preemption**: a resource can only be released voluntarily by the process holding it
- **circular wait**: a cycle of process requests exists (i.e., $P_0 \dashrightarrow P_1 \dashrightarrow P_2 \dashrightarrow \ldots \dashrightarrow P_{n-1} \dashrightarrow P_0$).

Example

Thread 1

```
void *do_work_one(void *param) {
  pthread_mutex_lock(&m1);
  pthread_mutex_lock(&m2);
  /**
   * Do some work
   */
  pthread_mutex_unlock(&m2);
  pthread_mutex_unlock(&m1);
  pthread_exit(0);
}
```

Thread 2

```
void *do_work_two(void *param) {
  pthread_mutex_lock(&m2);
  pthread_mutex_lock(&m1);
  /**
   * Do some work
   */
  pthread_mutex_unlock(&m1);
  pthread_mutex_unlock(&m2);
  pthread_exit(0);
}
```

Example

Will deadlock happen?

Thread 1

```
void *do_work_one(void *param) {
  pthread_mutex_lock(&m1);
  pthread_mutex_lock(&m2);
  /**
   * Do some work
   */
  pthread_mutex_unlock(&m2);
  pthread_mutex_unlock(&m1);
  pthread_exit(0);
}
```

Thread 2

```
void *do_work_two(void *param) {
  pthread_mutex_lock(&m2);
  pthread_mutex_lock(&m1);
  /**
   * Do some work
   */
  pthread_mutex_unlock(&m1);
  pthread_mutex_unlock(&m2);
  pthread_exit(0);
}
```

Example

Only if executed in order:
- *1, 3, 2, 4; or*
- *1, 3, 4, 2; or*
- *3, 1, 2, 4; or*
- *3, 1, 4, 2*

Thread 1

```
void *do_work_one(void *param) {
1 pthread_mutex_lock(&m1);
2 pthread_mutex_lock(&m2);
  /**
   * Do some work
   */
  pthread_mutex_unlock(&m2);
  pthread_mutex_unlock(&m1);
  pthread_exit(0);
}
```

Thread 2

```
void *do_work_two(void *param) {
3 pthread_mutex_lock(&m2);
4 pthread_mutex_lock(&m1);
  /**
   * Do some work
   */
  pthread_mutex_unlock(&m1);
  pthread_mutex_unlock(&m2);
  pthread_exit(0);
}
```

These orderings are ok:
- *1, 2, 3, 4; and*
- *3, 4, 1, 2*

Resource Allocation Graph

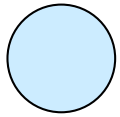
- A set of vertices **V** and a set of edges **E**

- **V** is partitioned into two types:
  - $P = \{P_1, P_2, \ldots, P_n\}$, the set of all the processes in the system
  - $R = \{R_1, R_2, \ldots, R_m\}$, the set of all resource types in the system

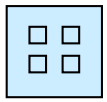
- **E** is partitioned into two types:
  - Request edge – directed edge $P_i \rightarrow R_j$
  - Assignment edge – directed edge $R_j \rightarrow P_i$

Resource Allocation Graph

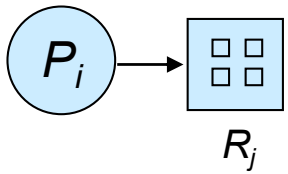
- Process



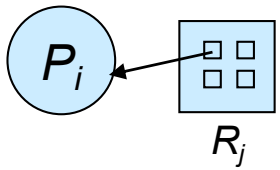
- Resource Type with 4 instances



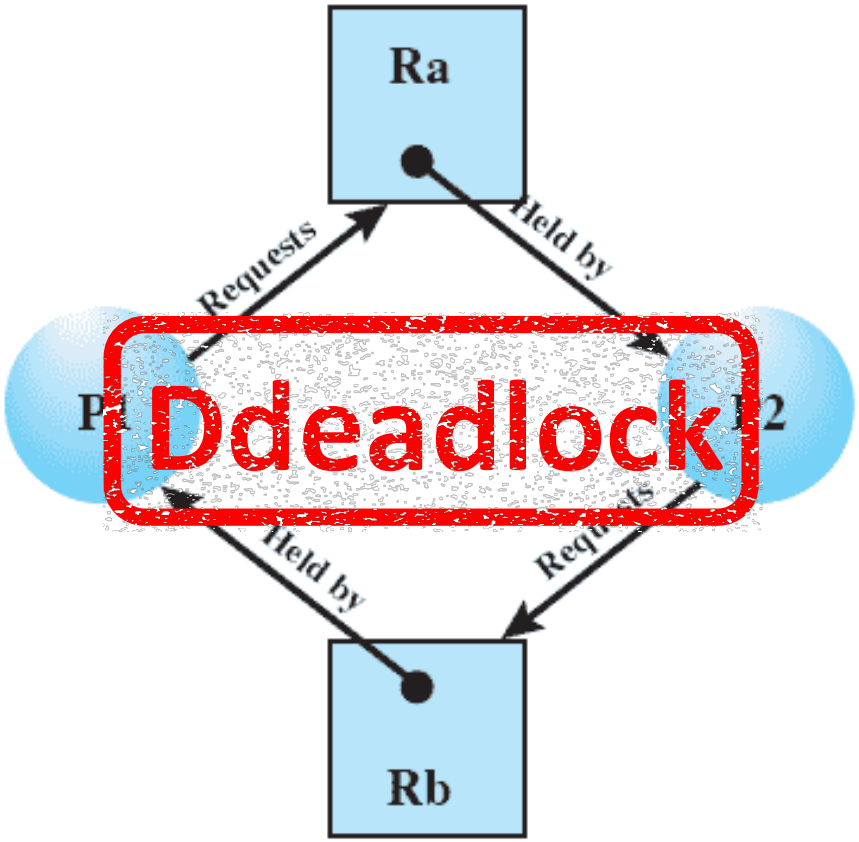
- P_i requests instance of R_j



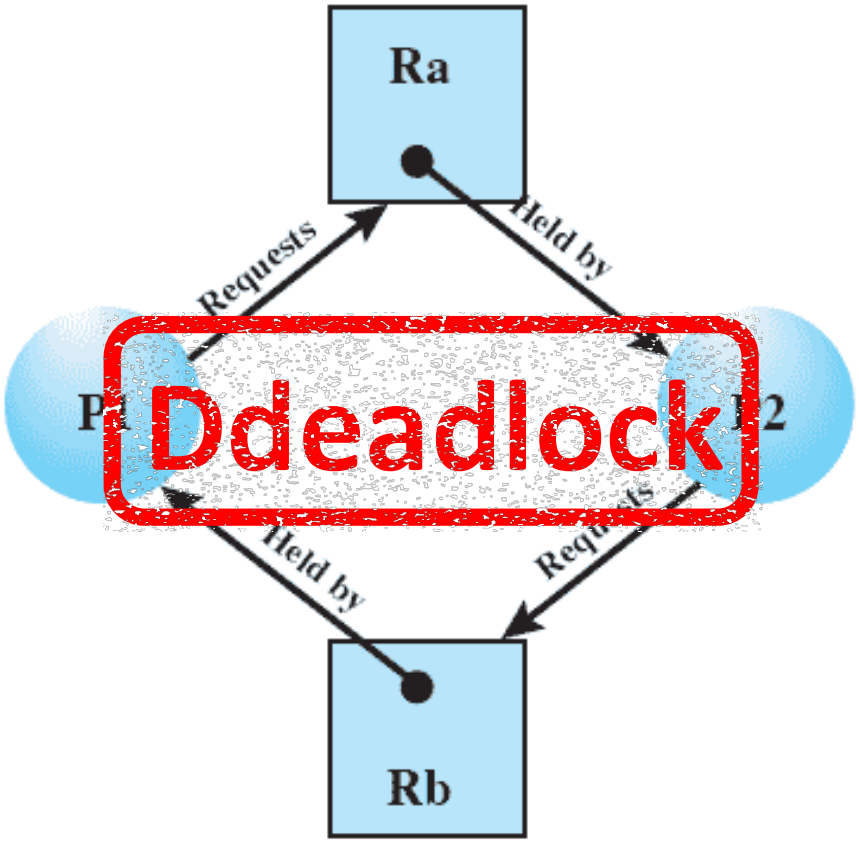
- P_i is holding an instance of R_j

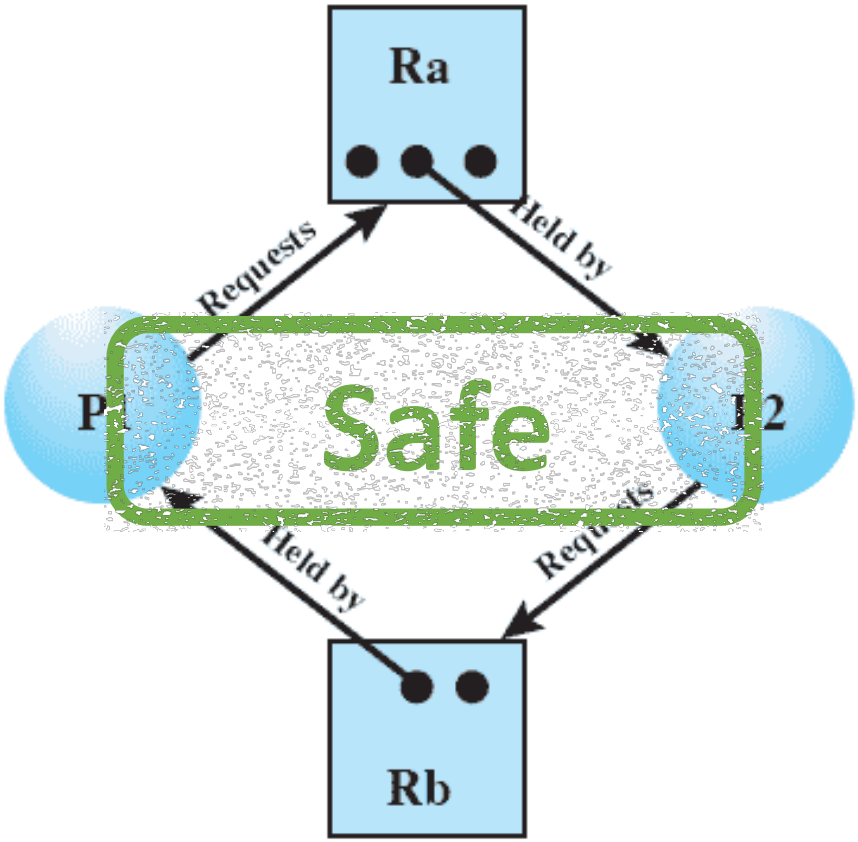


Example of Resource Allocation Graphs

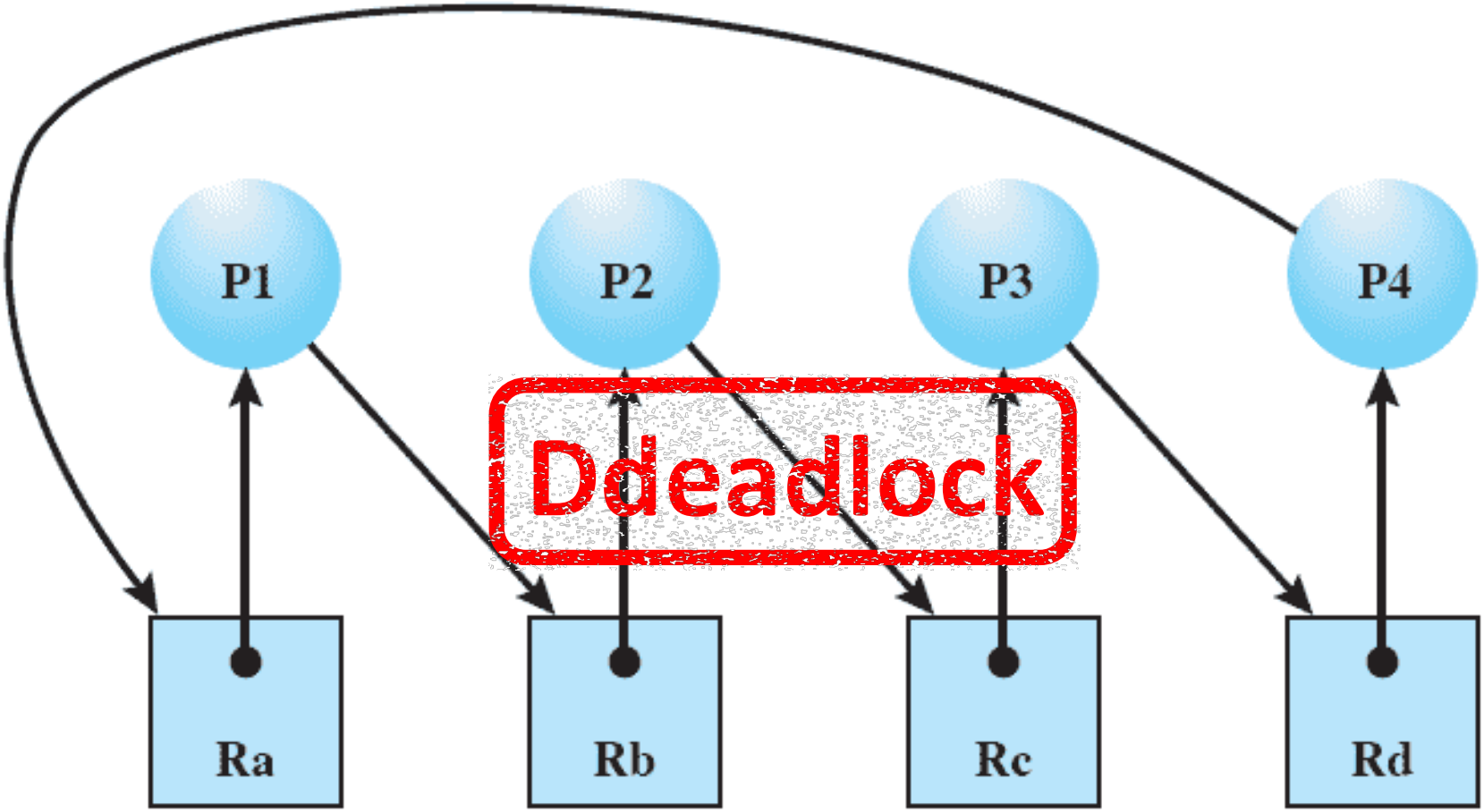


Example of Resource Allocation Graphs

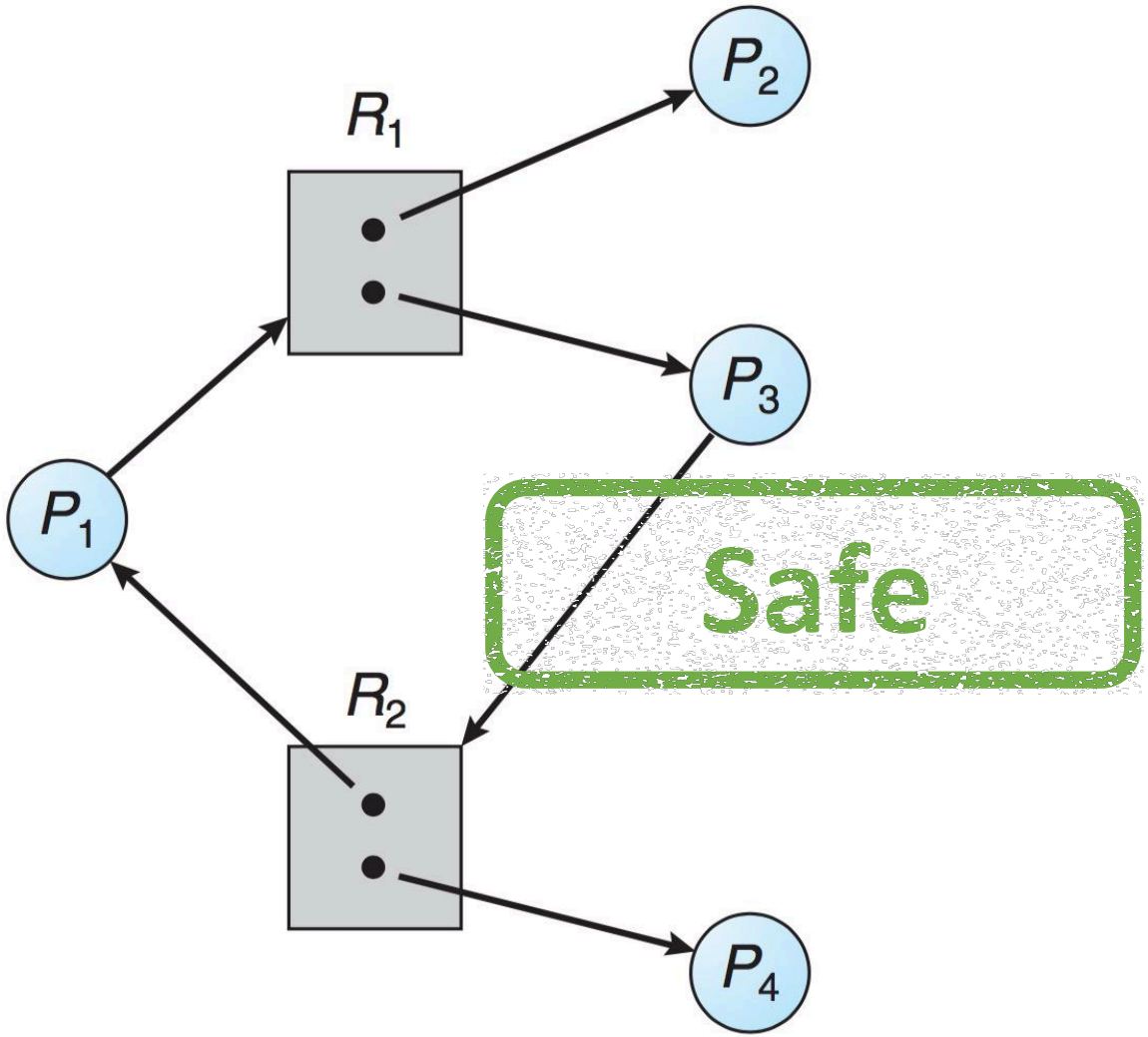




Example of Resource Allocation Graphs



Example of Resource Allocation Graphs



Basic Facts

- If graph contains no cycles $\Rightarrow$ no deadlock

- If graph contains a cycle $\Rightarrow$
  - if only one instance per resource type, then deadlock
  - if several instances per resource type, possibility of deadlock

How to Deal with Deadlocks?

- Deadlock prevention
- Deadlock avoidance

The system never enters a deadlock state

How to Deal with Deadlocks?

- Deadlock prevention
- Deadlock avoidance

The system never enters a deadlock state

- Deadlock detection and recovery
- Ignore the issue! 😇

The system may enter a deadlock state